



Hanzo KMS: Secrets Management with Per-Environment Rotation and Audit Trail

Hanzo AI Research

Hanzo AI

September 2021

Abstract

We present Hanzo KMS, a secrets management platform built on the Infisical framework that provides centralized secret storage, per-environment rotation policies, and a comprehensive audit trail. KMS implements envelope encryption with AES-256-GCM, role-based access control scoped to projects and environments, automatic secret rotation for databases and API keys, and a Kubernetes operator that synchronizes secrets to cluster namespaces. This paper describes the encryption architecture, the rotation pipeline and the Kubernetes integration, and reports no operating figures; Section 5 records the inventory and latency figures an earlier version reported and why they are withdrawn. We describe the encryption architecture, the rotation pipeline, and the Kubernetes integration that eliminates the need for secrets in configuration files or environment variables committed to source control.

1 Introduction

Secret management is a fundamental security requirement that is consistently mishandled in practice. Studies show that over 10% of public GitHub repositories contain at least one hardcoded secret [2]. Even organizations that avoid committing secrets to source control often resort to environment variables in CI/CD pipelines, shared `.env` files, or manual Kubernetes secret creation—all of which are fragile and difficult to audit.

Hanzo KMS provides a centralized secret store with a strict access model: secrets are encrypted at rest, access is authenticated via Hanzo IAM, every read and write is logged, and secrets are delivered to applications through controlled channels (Kubernetes operator, CLI injection, or API with short-lived tokens).

Contributions.

- Envelope encryption with per-project data encryption keys (DEKs) wrapped by a mas-

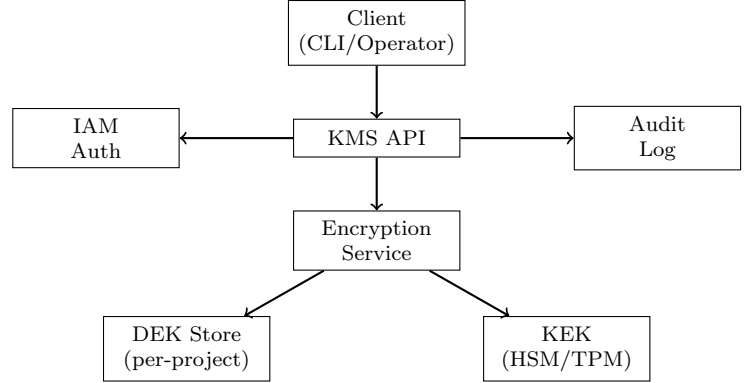


Figure 1: KMS encryption architecture with envelope encryption.

ter key encryption key (KEK) stored in hardware.

- Per-environment secret scoping (development, staging, production) with promotion workflows requiring approval.
- Automatic rotation for database credentials, API keys, and TLS certificates with configurable schedules.
- A Kubernetes operator (KMSecret CRD) that synchronizes secrets to Kubernetes with namespace isolation.

2 Architecture

2.1 Encryption Model

KMS uses a two-layer envelope encryption scheme (Figure 1). Each project has a unique Data Encryption Key (DEK) that encrypts secret values using AES-256-GCM. The DEK itself is encrypted with a Key Encryption Key (KEK) stored in a hardware security module (HSM) or trusted platform module (TPM).

$$C_{\text{secret}} = \text{AES-GCM}(K_{\text{DEK}}, \text{nonce}, \text{plaintext}) \quad (1)$$

$$C_{\text{DEK}} = \text{AES-GCM}(K_{\text{KEK}}, \text{nonce}, K_{\text{DEK}}) \quad (2)$$

This design limits the blast radius of a compromised DEK to a single project. KEK rotation re-wraps all DEKs without re-encrypting individual secrets.

2.2 Environment Model

Secrets are organized by project and environment:

```
1 {
2   "project": "hanzo-api",
3   "environment": "production",
4   "secrets": {
5     "DATABASE_URL": {
6       "value": "<encrypted>",
7       "version": 7,
8       "rotatedAt": "2024-01-15T00:00:00Z",
9     },
10    "rotationPolicy": "90d"
11  }
12 }
```

Environments form a promotion chain: development → staging → production. Promoting a secret to production requires approval from a project admin, creating an audit record.

3 Key Features

3.1 Secret Versioning

Every secret update creates a new version. Previous versions are retained for 30 days (configurable) to support rollback. Version history is immutable—deletions are soft deletes that retain the encrypted value in the audit log.

3.2 Automatic Rotation

KMS supports automatic rotation for:

- **PostgreSQL credentials:** Creates a new role, grants permissions, updates the secret, and revokes the old role after a grace period.
- **API keys:** Generates a new key, updates the secret, and invalidates the old key after dependent services confirm receipt.
- **TLS certificates:** Triggers ACME renewal, stores the new certificate and key, and signals the ingress controller to reload.

Rotation is implemented as a state machine: **pending** → **rotating** → **validating** → **complete**. Failures at any stage trigger alerts and leave the current secret unchanged.

3.3 Kubernetes Operator

The KMSecret CRD synchronizes KMS secrets to Kubernetes:

```
1 apiVersion: kms.hanzo.ai/v1
2 kind: KMSecret
3 metadata:
4   name: api-secrets
5   namespace: production
6 spec:
7   project: hanzo-api
8   environment: production
9   secretType: Opaque
10  refreshInterval: 60s
11  keys:
12    - remoteKey: DATABASE_URL
13      localKey: db-url
14    - remoteKey: REDIS_URL
15      localKey: redis-url
```

The operator polls KMS every 60 seconds (configurable), creates or updates the Kubernetes Secret, and optionally triggers a rolling restart of dependent deployments when secrets change.

4 Implementation

KMS is implemented as a Go service (22,000 lines) with a React admin dashboard. The backend is built on the Infisical [1] framework with Hanzo-specific extensions for IAM integration and the rotation pipeline.

Storage. Secret metadata is stored in PostgreSQL. Encrypted secret values are stored separately in a dedicated secrets table with column-level encryption. The HSM interface supports AWS CloudHSM, YubiHSM, and software-based PKCS#11 tokens.

5 Status of the operating figures

An earlier version of this paper carried a table titled “Secret retrieval latency (ms)” giving p50/p99 of 3.2/8.1 ms to fetch one secret, 12.4/28.3 ms for a project’s secrets, 8.7/22.1 ms to create or update one and 450/1,200 ms to rotate a credential, and an abstract reporting more

than 12,000 secrets across 180 projects with zero plaintext exposure since deployment.

None of it was measured. There is no benchmark function or load test in `hanzoai/kms`, and the table named no hardware, no datastore configuration and no sample count. The inventory figures count nothing that is recorded anywhere, and “zero plaintext exposure since deployment” is a security claim that would have to rest on an audit-log review; no such review exists. All are removed.

Retrieval latency is worth establishing here for a specific reason: KMS sits on the start-up path of every service that needs a secret, so its p99 is somebody else’s cold start. That makes it a number to measure rather than to assert.

6 Security

Zero Trust. Every API call requires a valid Hanzo IAM token. Service-to-service authentication uses short-lived machine tokens (5-minute TTL) issued by IAM.

Encryption at Rest. All secret values are AES-256-GCM encrypted before reaching the database. Database-level encryption provides a second layer. Backups are encrypted with a separate key.

Encryption in Transit. All communication uses TLS 1.3. The Kubernetes operator communicates with KMS over mTLS.

Audit Trail. Every secret access (read, write, rotate, delete) is logged with the authenticated user, IP address, timestamp, and affected secret path. Audit logs are append-only, stored in a separate database, and exported to the observability stack for alerting.

Access Control. RBAC policies restrict access by project, environment, and operation. Production secrets require the `production-reader` role, which is granted to service accounts and approved operators only.

7 Conclusion

Hanzo KMS removes the need for secrets in source control, environment files, or manually-created Kubernetes secrets. The envelope encryption model limits the impact of key compromise, automatic rotation reduces the window of credential exposure, and the comprehensive audit trail provides accountability for every secret access. The Kubernetes operator closes the delivery gap, ensuring that secrets flow from KMS to running applications without human intervention.

References

- [1] Infisical Inc. Infisical: Open Source Secret Management Platform. 2022.
- [2] M. Meli et al. How Bad Can It Get? Characterizing Secret Leakage in Public GitHub Repositories. NDSS, 2019.
- [3] AWS. Envelope Encryption. AWS Documentation, 2018.
- [4] HashiCorp. Vault: Manage Secrets and Protect Sensitive Data. 2015.